# Slide 1 — Week 3

Database Implementation

Focus:

- Implement a relational schema — in DuckDB, then MySQL
- Data types and constraints
- Loading and validating data
- Indexes and query performance
- Transactions, ACID and concurrency

# Slide 2 — From Design to Implementation

Week 1

ER modeling

Week 2

Relational schema design

Week 3

Database implementation

# Slide 3 — Implementation Workflow

Typical workflow

1. Create database
2. Create tables
3. Define constraints
4. Add indexes
5. Load data

# Slide 4 — Two Engines

**DuckDB** — embedded, in-process, zero setup

- runs inside your notebook; no server
- optimized for analytical queries
- the engine for every lab

**MySQL** — client–server, multi-user

- a separate server many applications connect to
- optimized for transactional workloads
- the production-system contrast

# Slide 5 — Creating a Database

MySQL:

```
CREATE DATABASE ecommerce;
USE ecommerce;
```

DuckDB — a database is a file:

```
con = duckdb.connect("ecommerce.duckdb")
```

# Slide 6 — Creating Tables

Example:

```sql
CREATE TABLE customers (
  customer_id INT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(200),
  country VARCHAR(50)
);
```

# Slide 7 — Data Types

Common data types

INTEGER

VARCHAR

DATE

DECIMAL

Example:

```
price DECIMAL(10,2)
```

Use DECIMAL, not DOUBLE, for money — floating point does not add cents exactly.

# Slide 8 — Orders Table

```sql
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  customer_id INT,
  order_date DATE,
  FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id)
);
```

# Slide 9 — Products Table

```sql
CREATE TABLE products (
  product_id INT PRIMARY KEY,
  name VARCHAR(100),
  category VARCHAR(100),
  price DECIMAL(10,2)
);
```

# Slide 10 — Order Items Table

```sql
CREATE TABLE order_items (
  order_item_id INT PRIMARY KEY,
  order_id INT,
  product_id INT,
  quantity INT,
  FOREIGN KEY (order_id) REFERENCES orders(order_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

# Slide 11 — Payments Table

```
CREATE TABLE payments (
  payment_id INT PRIMARY KEY,
  order_id INT,
  amount DECIMAL(10,2),
  payment_date DATE,
  FOREIGN KEY (order_id)
    REFERENCES orders(order_id)
);
```

# Slide 12 — Constraints

Constraints enforce data integrity.

Examples

PRIMARY KEY

FOREIGN KEY

NOT NULL

UNIQUE

CHECK

# Slide 13 — Example NOT NULL

```
email VARCHAR(200) NOT NULL
```

Ensures every row contains an email.

# Slide 14 — Unique Constraint

```
email VARCHAR(200) UNIQUE
```

Prevents duplicate email addresses.

# Slide 15 — Indexes

Indexes improve query performance.

Example

```
CREATE INDEX idx_customer
ON orders(customer_id);
```

# Slide 16 — Why Indexes Matter

Row store (MySQL)

Without index → full table scan
With index → fast lookup

Columnar store (DuckDB)

Zone maps (min/max per block) already skip data —
an index on a sorted or low-cardinality column can buy nothing, or slow a scan.

**Measure, don't assume.**

# Slide 17 — Loading Data

Basic insertion

```
INSERT INTO customers
VALUES (1,'Alice','alice@email.com','USA');
```

# Slide 18 — Bulk Data Loading

MySQL

```
LOAD DATA LOCAL INFILE 'customers.csv'
INTO TABLE customers
FIELDS TERMINATED BY ','
IGNORE 1 ROWS;
```

DuckDB

```
INSERT INTO customers
SELECT * FROM read_csv('customers.csv');
```

# Slide 19 — Validating the Load

Never assume a load worked.

- compare row counts with the source
- compare a control total (e.g. SUM(amount))
- cast messy values with TRY_CAST and quarantine the rows that fail
- make reloads idempotent — running twice must not duplicate rows

# Slide 20 — Duplicate Business Keys

The source sends the **same supplier twice** — with different values:

| supplier_code | supplier_name | city | updated_at |
|---|---|---|---|
| S-01 | Acme Parts | Fresno | 2026-02-01 10:00 |
| S-01 | Acme Parts Inc. | Fresno | 2026-02-03 09:00 |
| S-02 | Bay Metals | *NULL* | 2026-02-02 08:00 |
| S-02 | Bay Metals | Oakland | 2026-02-02 08:00 |
| S-03 | Coast Tools | Salinas | 2026-02-01 12:00 |

- `SELECT DISTINCT` keeps all **5** rows — they aren't identical
- a primary key on `supplier_code` rejects the load
- you need a **survivorship rule**: which row wins?

# Slide 21 — A Survivorship Rule in SQL

Write the rule down, in order — then encode it with `ROW_NUMBER()`:

```sql
SELECT supplier_code, supplier_name, city, updated_at
FROM (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY supplier_code
               ORDER BY updated_at DESC,    -- 1. newest wins
                        (city IS NULL),     -- 2. then a known city
                        supplier_name       -- 3. then a fixed tie-break
           ) AS rn
    FROM stg_supplier
) AS ranked
WHERE rn = 1;
```

Result: **3 rows** — S-01 *Acme Parts Inc.*, S-02 *Oakland*, S-03.

- The last tie-break makes it **deterministic**: same input → same output, every run

# Slide 22 — Testing the Database

Example query

```
SELECT c.country,
       COUNT(o.order_id)
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.country;
```

# Slide 23 — Transactions and ACID

A transaction groups steps into one all-or-nothing unit.

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;   -- or ROLLBACK if anything failed
```

**A**tomicity · **C**onsistency · **I**solation · **D**urability

# Slide 24 — Concurrency

Two users update the same row at the same time.

Without care: a **lost update**.

MySQL (InnoDB) — locks the row; the second writer **waits**

DuckDB — optimistic; the second writer gets a **"Conflict on update!"** error and must **retry**

# Slide 25 — Implementation Checklist

Database created

Tables created

Keys and constraints defined

Data loaded and validated

Indexes measured, not assumed

Multi-step changes wrapped in transactions

Queries tested

# Slide 26 — Week 3 Labs

Each session has a core lab (**a**) and a challenge lab (**b**), started in the last ~40 minutes of class and submitted on Camino by 11:00 pm Pacific Time.

**Session 5 — Typed DDL and loading messy data**

- Lab 1a — Copperline Coffee: typed DDL, TRY_CAST quarantine, reconciliation
- Lab 1b — Harborline Provisions: DECIMAL vs DOUBLE, an idempotent load, set-based vs row-at-a-time

**Session 6 — Transactions and concurrency**

- Lab 2a — Sablewood Credit Union: autocommit, ROLLBACK, COMMIT, two writers on one row
- Lab 2b — Cascade Rail: snapshot isolation, write-write conflicts, retries, SQL injection